





Advanced ER Modelling and ER-to-Relational Mapping

Bridging Real World Requirements and Relational Database Design



Learning Outcomes

- By the end of our session, we will be able to:
 - Understand advanced ER concepts.
 - Explain specialization and generalization.
 - Differentiate between aggregation and composition (conceptually).
 - Understand inheritance in EER models.
 - Convert an ER diagram into relational tables.
 - Apply mapping rules to real-world databases.



haribansh@bhu.ac.in

Why Do We Need Advanced ER Modelling?

- The basic ER model is sufficient for simple databases.
- However, real-world applications often involve:
 - Different categories of entities
 - Hierarchical relationships
 - Shared properties
 - Complex business rules
- Examples
 - University
 - *Faculty*
 - Permanent Faculty
 - Visiting Faculty
 - Hospital
 - *Doctor*
 - Surgeon
 - Physician
 - Bank
 - *Account*
 - Savings Account
 - Current Account
- These situations require an Enhanced ER (EER) Model.



haribansh@bhu.ac.in

Enhanced Entity-Relationship (EER) Model

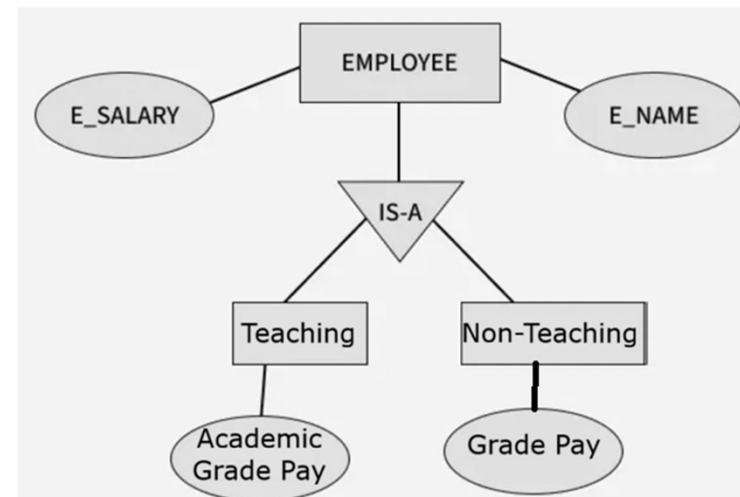
- Definition
- The Enhanced Entity-Relationship (EER) Model extends the basic ER model by introducing concepts such as:
 - Specialization
 - Generalization
 - Inheritance
 - Categories (Union Types)
 - Aggregation
- *Advantages*
- Represents complex real-world systems.
- Reduces redundancy.
- Improves database design.
- Supports object-oriented concepts.
- *Note:* Most modern database design tools support EER modelling.



haribansh@bhu.ac.in

Specialization

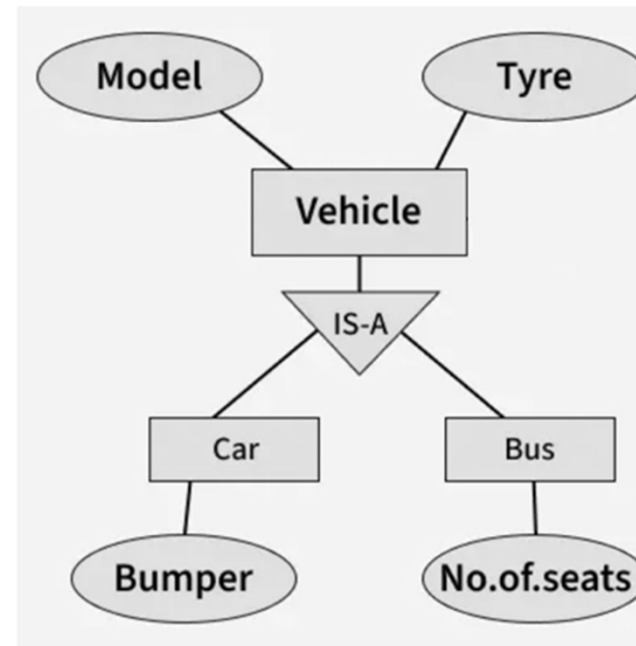
- Definition
- Specialization is the process of dividing one higher-level entity into smaller specialized entities based on distinguishing characteristics.
- *Example*
- Employee → Teaching Staff
- Employee → Non-Teaching Staff
- Both inherit common employee information.
- *Characteristics*
- Top-down approach
- Creates subclasses
- Supports inheritance
- *Example Attributes*
 - Employee
 - Employee ID
 - Name
 - Salary
 - Teaching Staff
 - Academic Pay Grade
 - Non-Teaching Staff
 - Pay Grade



haribansh@bhu.ac.in

Generalization

- Definition
- Generalization is the process of combining similar lower-level entities into a higher-level entity.
- *Example*
Car, Truck, Bus
↓
Vehicle
- *Characteristics*
- Bottom-up approach
- Combines common features
- Eliminates redundancy
- *Comparison*
 - Specialization
 - *One* → *Many*
 - Generalization
 - *Many* → *One*



Specialization vs Generalization

- *Specialization*

- *Generalization*

• =====

- Top-down approach
- Creates subclasses
- More specific entities
- Employee → Faculty, Clerk

- Bottom-up approach
- Creates superclass
- More generalized entity
- Car, Bus → Vehicle

- *Memory Trick*
- Specialization
- "Divide" → Specific
- Generalization
- "Combine" → General



Inheritance in EER Model

- Definition
- Inheritance means that a subclass automatically acquires the properties of its superclass.
- *Example*
- Employee
 - Employee ID
 - Name
 - Salary
- Faculty inherits
 - Employee ID
 - Name
 - Salary
 - and adds*
 - Subject
 - Qualification
- *Benefits*
- Avoids repetition
- Simplifies design
- Improves consistency



haribansh@bhu.ac.in

Constraints in Specialization

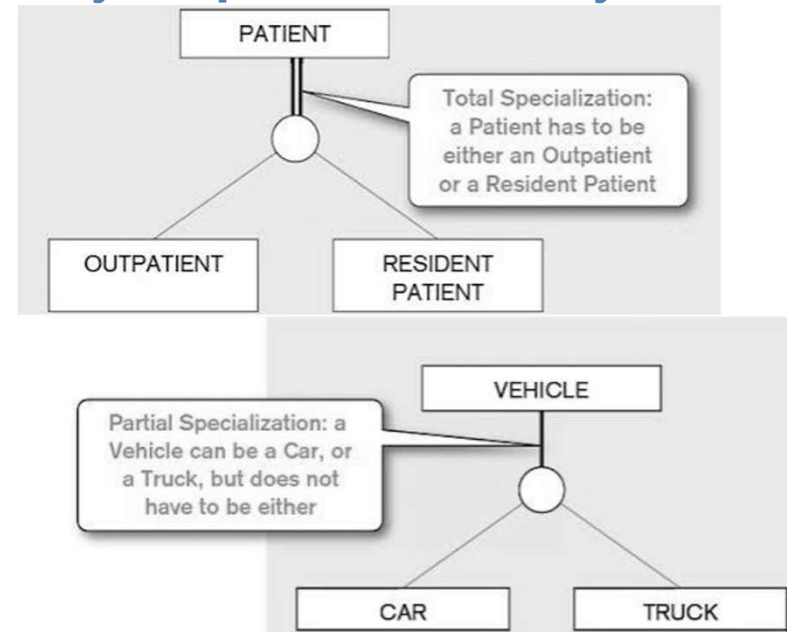
- Two important constraints exist.
- *Disjoint Constraint*
- An entity belongs to only one subclass.
- *Example*
- Employee is either
 - Faculty
 - OR
 - Administrator
- *Overlapping Constraint*
- An entity may belong to multiple subclasses.
- *Example*
- Faculty member
 - Teacher
 - Researcher
- The same person performs both roles.



haribansh@bhu.ac.in

Completeness Constraints

- Completeness specifies whether every superclass entity must belong to a subclass.
- *Total Specialization*
- Every Employee must be
 - Teaching
 - OR
 - Non-Teaching.
- Represented by: Double Line
- *Partial Specialization*
- Some Employees may not belong to any subclass.
- Represented by: Single Line
- Example: Employee: Researcher, Teacher; But we can have others like Librarians, Security Staff...



haribansh@bhu.ac.in

Aggregation

- Definition
- Aggregation is an abstraction that treats a relationship as a higher-level entity.
- It is used when a relationship itself participates in another relationship.

- *Example*

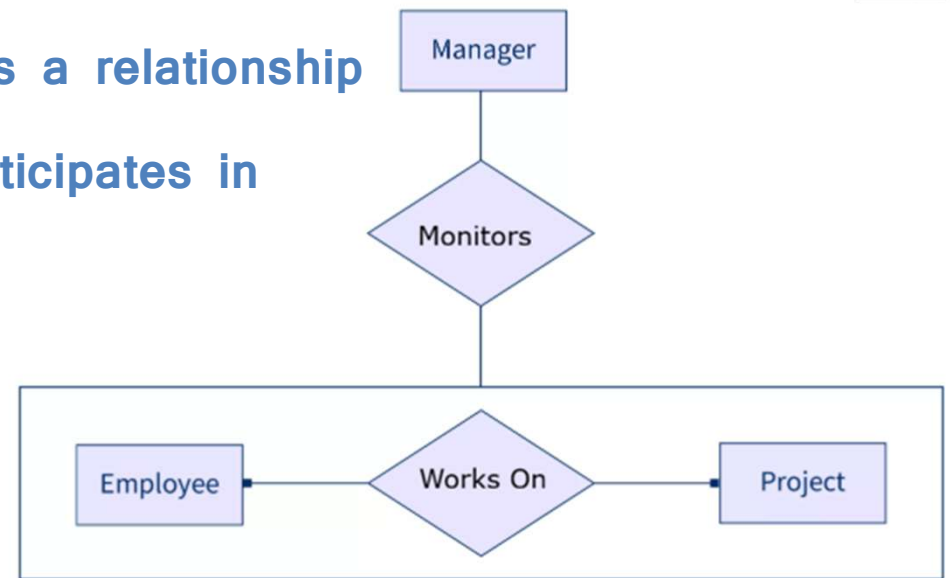
Employee → Works On → Project



- Manager monitors this assignment.
- Instead of connecting the manager directly, the relationship "Works On" is aggregated.

- *Application*

- Project Management Systems
- Manufacturing
- ERP Systems



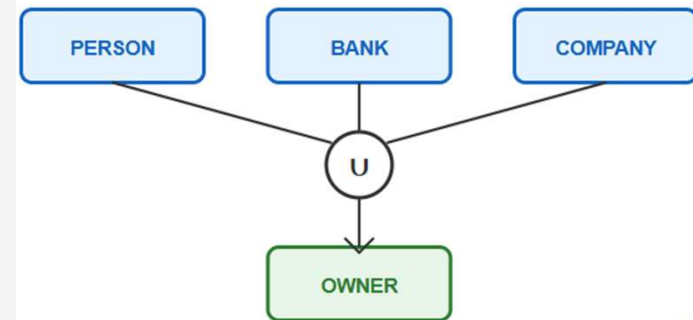
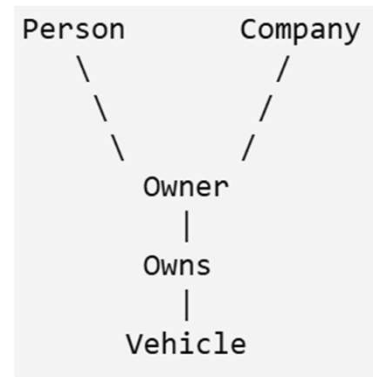
haribansh@bhu.ac.in

Categories (Union Types)

- Sometimes a subclass is formed from multiple superclasses.

- *Example*

- Owner
can be
- Person
OR
- Company



haribansh@bhu.ac.in

ER-to-Relational Mapping

- *Why Mapping?*
- ER diagrams are conceptual but RDBMS store information as tables.
- Therefore, ER diagrams must be converted into relational schemas.
- *General Rule*
- Entity → Table
- Attribute → Column
- Entity Instance → Row
- Relationship → Foreign Key or New Table



haribansh@bhu.ac.in

Mapping Strong Entities

- Rule
- Each strong entity becomes one table.
- *Example*
- Entity: Student
- Attributes:
 - Roll No
 - Name
 - Email
- *Table*
- Student
- | RollNo (PK) | Name | Email |



Mapping Weak Entities

- Rule
- Weak entities become separate tables.
- Their primary key includes
- Partial Key
- Primary Key of the owner entity
- *Example*
- Employee → Dependent
- Dependent
 - EmpID
 - Dependent Name
 - Age
- Primary Key
 - (EmpID, DependentName)



haribansh@bhu.ac.in

Mapping Relationships

- One-to-One (1:1)
 - Add foreign key to either table.
 - Example: Person – Passport
- One-to-Many (1:M)
 - Add foreign key to the "Many" side.
 - Example: Department → Student
 - DepartmentID becomes a foreign key in Student.
- Many-to-Many (M:N)
 - Create a new relation.
 - Example: Student → Enrolls → Course
 - Enrollment
 - | RollNo | CourseID |
 - Composite Primary Key
 - (RollNo, CourseID)



haribansh@bhu.ac.in

Mapping Multi-Valued Attributes

- Rule
- Create a separate relation.
- *Example*
- Student
 - Phone Numbers
- StudentPhone
- | RollNo | Phone |
- Primary Key
- (RollNo, Phone)
- *Why?*
- A relational table should not store multiple values in one column.



Modern Perspective

- Modern databases still begin with conceptual Modelling.
- Relational Databases
- ER Diagram → Tables
- MongoDB
- ER concepts become Collections, Documents, Embedded Documents, References
- Graph Databases (Neo4j)
- Entities → Nodes
- Relationships → Edges
- Properties remain attributes.
- *Important* The Modelling concepts remain valuable even when the storage model changes.



Summary

- We learned
- ✓ Enhanced ER Model
- ✓ Specialization
- ✓ Generalization
- ✓ Inheritance
- ✓ Disjoint Constraints
- ✓ Overlapping Constraints
- ✓ Total and Partial Specialization
- ✓ Aggregation
- ✓ Categories (Union Types)
- ✓ ER-to-Relational Mapping
- ✓ Mapping Strong Entities
- ✓ Mapping Weak Entities
- ✓ Mapping Relationships
- ✓ Mapping Multi-Valued Attributes



Learning Reflections

- Why was the EER model introduced?
- Differentiate between specialization and generalization.
- Explain inheritance with a suitable example.
- Differentiate between disjoint and overlapping specialization.
- Explain total and partial specialization.
- What is aggregation? When is it used?



haribansh@bhu.ac.in

Homework

- Discuss specialization, generalization, inheritance, disjoint and overlapping constraints, total and partial specialization, and aggregation with suitable examples. Also explain the basic ER-to-relational mapping rules for strong and weak entities, 1:1, 1:M and M:N relationships, and multi-valued attributes, with suitable examples.





haribansh@bhu.ac.in

Thanks!

📌 Heads Up!

These slides are just a **guide** — not the full story!

To really learn, make sure to read your book, and explore other resources too.